
上海健康医学院

实验报告

院（系）： 医疗器械学院

课 程： 医疗大数据统计

专业班级： 22 级数据科学与大数据技术 2 班

学 号： B22040302020

学生姓名： 周雨晴

指导教师： 郭家辰

任务列表

实验任务：拓展感兴趣知识（思考性实验）

实验任务一：t 检验（验证性实验）

预习任务：t 检验&方差检验（预习）

实验任务二：方差分析（验证性实验）

实验任务三：正态分布（验证性实验）

实验任务四：秩和检验（一）（验证性实验）

实验任务五：秩和检验（二）（验证性实验）

实验任务六：相关系数检验（验证性实验）

实验任务七：生存分析（一）（验证性实验）

实验任务八：生存分析（二）（验证性实验）

实验任务

院（系）：医疗器械学院

日期：2023.9.22

拓展感兴趣知识（思考性实验）

实验任务：

对第一节课上课内容，自己所感兴趣的部分进行拓展。

实验工具：

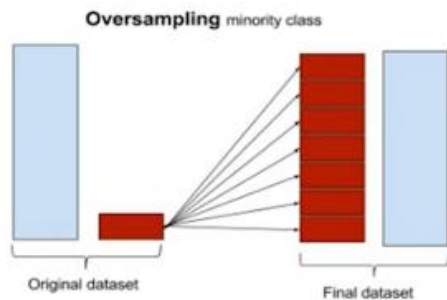
电脑、Internet

实验内容：**数据不平衡分类**

- 不平衡分类问题是指训练样本数量在类间分布不平衡的模式分类问题。
【即：大部分数据是正常的用户，只有少部分数据是生病的用户】

【解决方法】——基于抽样的方法**过采样 oversampling****【定义】**

• 过采样，会增加训练集中少数群体成员的数量。过采样的优点是保留原始训练集中的信息，因为会保留少数和多数类别的所有观察结果。另一方面，它容易过度拟合。

**【实操】**

- 复制正样本，直到训练集中正样本和负样本一样多

【问题】

• 可能导致模型过分拟合，因为一些噪声样本也可能被复制多次（噪声越多，过拟合越高）

【解决方法】

对关心的正例（即生病的用户）进行有放回的抽样：

训练集可能规模很小，正样本通过有放回的抽样，将其又放回的抽样，将训

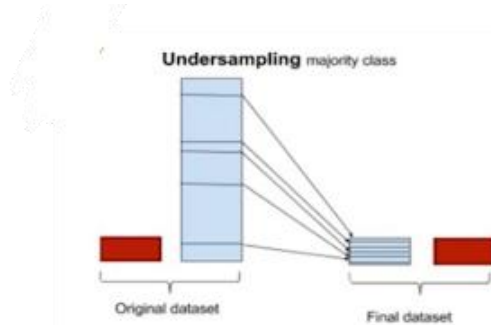
练集不断扩大；

将生病用户与之前的正常用户的数据量大体保持一致

欠采样 undersampling

【定义】

• 欠采样，与过采样相反，旨在减少多数样本的数量来平衡类分布；由于它正在从原始数据集中删除观察结果，因此可能会丢弃有用的信息。



【实操】

- 随机抽取 100 个负样本，与所有的正样本一样形成训练集

【问题】

- 一些有用的负样本可能没有选出来用于训练，因此导致一个不太优的模型

【解决方法】

- 多次执行不充分抽样，并归纳类似于组合学习方法的多分类器

实训总结：

【R 代码实现】

• 在 R 语言中，*ROSE (Random Over Sampling Examples)* 包用于处理样本不平衡问题。

• ROSE 包中内置了一个叫做 *hacide* 的不平衡数据集，它包括 *hacide.train* 和 *hacide.test* 两个部分

• *cls* 是响应变量，*x1* 和 *x2* 是解释变量

• ROSE 包提供了名为 *accuracy.meas()* 的函数，它能用来计算准确率，召回率和 F 测度等统计量。

• 如果设定阈值为 0.5，准确率等于 1 说明没有被误分为正类的样本。召回率等于 0.2 意味着有很多样本被误分为负类。0.167 的 F 值也说明模型整体精度很低

• 使用 *roc.curve()* 函数可以绘制模型的 ROC 曲线

• *ovun.sample()* 的函数来实现过采样和欠采样

• 同时采取这两类方法，只需要把参数改为 *method = "both"*

实验任务一

院（系）：医疗器械学院

日期：2023.10.13

t 检验（验证性实验）
实验任务： 1、掌握单样本t检验 2、掌握配对样本t检验 3、掌握两独立样本t检验 4、运行结果展示和解释
实验工具： 电脑、Internet、R、Python
实验过程： 设置目录 <pre>setwd("E:\\sophomore\\subject\\医疗大数据统计\\Data\\7")</pre> 1. 7-1 运行代码及结果： <pre>> #7-1 > mean1<-6.76*10^9 > sd1<-1.36*10^9 > n1<-36 > alpha<-0.05 > t1<-qt(1-alpha/2,n1-1,lower.tail =TRUE) > t1 [1] 2.030108 > c(mean1-sd1*t1/sqrt(n1),mean1+sd1*t1/sqrt(n1)) [1] 6299842203 7220157797</pre>
2. 7-2 运行代码及结果： <pre>> #7-2 > mean2<-172.2 > sd2<-4.5 > n2<-90 > alpha<-0.05 > z1<-qnorm(1-alpha/2,mean=0,sd=1,lower.tail = TRUE) > z1 [1] 1.959964 > c(mean2-sd2*z1/sqrt(n2),mean2+sd2*z1/sqrt(n2)) [1] 171.2703 173.1297</pre>

3. 7-3

运行代码及结果:

```
> #7-3
> mean0<-4*10^9
> alpha<-0.05
> t2<-(mean1-mean0)/(sd1/sqrt(n1))
> t2
[1] 12.17647

> qt(1-alpha,n1-1,lower.tail =TRUE)
[1] 1.689572

> 1-pt(t2,df=35)
[1] 1.931788e-14
```

4. 7-4

运行代码及结果:

```
> #7-4
> data7.4<-read.csv("7-4.csv")
> t.test(data7.4$sq,data7.4$sh,paired=TRUE)$conf.int
[1] 5.630050 8.204495
attr(,"conf.level")
[1] 0.95
```

5. 7-5

运行代码及结果:

```
> #7-5
> a<-t.test(data7.4$sq,data7.4$sh,paired=TRUE)
> a$statistic
      t
11.97357

> alpha2<-0.001
> n3<-11
> qt(1-alpha2/2,n3-1,lower.tail =TRUE)
[1] 4.586894

> a$p.value
[1] 2.982617e-07
```

6. 7-6

运行代码及结果:

```
> #7-6
> mean4<-27.2
> sd4<-0.9
> n4<-44
> mean5<-27.3
> sd5<-0.8
> n5<-48
> t4<-qt(1-alpha/2,n4+n5-2,lower.tail =TRUE)
> t4
[1] 1.986675

> sw<-sqrt((((n5-1)*sd5^2+(n4-1)*sd4^2)/(n4+n5-2))*(1/n5+1/n4))
> c((mean5-mean4)-t4*sw,(mean5-mean4)+t4*sw)
[1] -0.2521342 0.4521342
```

7. 7-7

运行代码及截图：

```
> #7-7
> mean6<-2.9
> sd6<-0.3
> n6<-10
> mean7<-2.8
> sd7<-0.1
> n7<-29
> sw2<-sqrt(sd6^2/n6+sd7^2/n7)
> sw2
[1] 0.09666865

> v<-(sd6^2/n6+sd7^2/n7)^2/((sd6^2/n6)^2/(n6-1)+(sd7^2/n7)^2/(n7-1))
> v
[1] 9.698291

> t5<-qt(1-alpha/2,round(v,0),lower.tail = TRUE)
> t5
[1] 2.228139
```

8. 7-8

运行代码及结果：

```
> #7-8
> data7.8<-read.csv("7-8.csv")
> colnames(data7.8)<-c("gdb","ddb")
> b<-t.test(data7.8$gdb,data7.8$ddb,var.equal = TRUE)
> b$statistic
      t
1.891436

> b$p.value
[1] 0.07573013

> c((mean6-mean7)-t5*sw2,(mean6-mean7)+t5*sw2)
[1] -0.1153912  0.3153912
```

9. 7-9

运行代码及结果：

```
> #7-9
> mean8<-0.345
> sd8<-0.053
> n8<-25
> mean9<-0.362
> sd9<-0.083
> n9<-15
> alpha=0.05
> tt<-(mean9-mean8)/sqrt(sd8^2/n8+sd9^2/n9)
> tt
[1] 0.7110378
```

```
> v1<-(sd8^2/n8+sd9^2/n9)^2/((sd8^2/n8)^2/(n8-1)+(sd9^2/n9)^2/(n9-1))
> v1
[1] 20.95649

> qt(1-alpha/2,round(v1,0),lower.tail= TRUE)
[1] 2.079614

> 1-pt(tt,round(v1,0))
[1] 0.2424421
```

10. 7-10

运行代码及结果:

```
> #7-10
> f<-sd6^2/sd7^2
> f
[1] 9

> alpha3<-0.10
> qf(1-alpha3,n6-1,n7-1,lower.tail = TRUE)
[1] 1.865199

> 1-pf(f,n6-1,n7-1,lower.tail =TRUE)
[1] 3.09081e-06
```

11. 7-13

运行代码及结果:

```
> #7-13
> total<-166
> yang<-41
> p<-yang/total
> p
[1] 0.246988

> z1<-qnorm(1-alpha/2,mean=0,sd=1,lower.tail =TRUE)
> c(p-z1*sqrt(p*(1-p)/total),p+z1*sqrt(p*(1-p)/total))
[1] 0.1813836 0.3125923
```

12. 7-14

运行代码及结果:

```
> #7-14
> total2<-8
> yang2<-5
> pp<-(yang2+2)/(total2+4)
> pp
[1] 0.5833333

> z1<-qnorm(1-alpha/2,mean=0,sd=1,lower.tail = TRUE)
> c(pp-z1*sqrt(pp*(1-pp)/(total2+4)),
+   pp+z1*sqrt(pp*(1-pp)/(total2+4)))
[1] 0.3043937 0.8622730
```


13. 7-15

运行代码及结果:

```
> #7-15
> p0<-0.0043
> total3<-500
> yang3<-16
> c<-binom.test(yang3,total3,p=p0,alternative = c( "greater"))
> c$p.value
[1] 1.106037e-09
```

14. 7-16

运行代码及结果:

```
> #7-16
> p01<-0.0739
> total4<-3909
> yang4<-1121
> d<-prop.test(yang4,total4,p01,alternative=c("greater"))
> sqrt(d$statistic)
X-squared
50.84444

> d$p.value
[1] 0
```

15. 7-17

运行代码及结果:

```
> #7-17
> total5<-1430
> yang5<-313
> p1<-yang4/total4
> p1
[1] 0.2867741

> p2<-yang5/total5
> p2
[1] 0.2188811

> z1<-qnorm(1-alpha/2,mean=0,sd=1,lower.tail =TRUE)
> c((p1-p2)-z1*sqrt(p1*(1-p1)/total4+p2*(1-p2)/total5),
+ (p1-p2)+z1*sqrt(p1*(1-p1)/total4+p2*(1-p2)/total5))
[1] 0.04219690 0.09358909
```

16. 7-18

运行代码及结果:

```
> #7-18
> total6<-4
> yang6<-3
> pp1<-(yang6+1)/(total6+2)
> pp1
[1] 0.6666667

> total7<-3
> yang7<-2
> pp2<-(yang7+1)/(total7+2)
> pp2
[1] 0.6
```

```

> z1<-qnorm(1-alpha/2,mean=0,sd=1,lower.tail =TRUE)
> sw1<-sqrt(pp1*(1-pp1)/(total6+2)+pp2*(1-pp2)/(total7+2))
> sw1
[1] 0.2916111

> c((pp1-pp2)-z1*sw1,(pp1-pp2)+z1*sw1)
[1] -0.5048806 0.6382139

```

17. 7-19

运行代码及结果:

```

> #7-19
> total8<-c(total4,total5)
> yang8<-c(yang4,yang5)
> e<-prop.test(yang8,total8)
> pc<-(yang4+yang5)/(total4+total5)
> pc
[1] 0.2685896

> e$p.value
[1] 8.584585e-07

```

实训总结:

\$:

- \$是面向对象的，表示从 dataframe 中取出某一列数据
- 当一个函数要返回多个值时，要提取某个变量的结果，就需要用到\$，对象通常是列表

qt 函数:

- qt 函数是用来计算 t 分布的一种函数，是 R 语言自带的基础库
- qt(p, df, ncp, lower.tail = TRUE, log.p = FALSE)
- p 是概率向量，df 是自由度，ncp 省略则是中心 t 分布，逻辑如果为 TRUE (默认值)，则概率为 $P[X \leq x]$ ，否则为 $P[X > x]$

P value:

- p value，代表在原假设条件下，实验事件可能发生的概率

参数 var.equal:

- 设置参数 var.equal=TRUE, 指定样本之间是等方差的

在 R 语言中，t 检验常用参数及其意义:

参数	意义
x	数据框，即自己要分析的数据
alternative	指定单侧还是双侧检验，"less", "greater"为单侧；"two.sided"为双侧
mu	指定与自己数据进行比较的常数
paired	是否为配对t检验，默认不是
var.equal	方差齐不齐，默认为FALSE
conf.level	置信区间，默认为95%

三种 t 检验代码示例：

```
pre <- c(6,10,3,5,7)
post <- c(1,2,0,5,2)

#检验数据是否服从正态分布
shapiro.test(pre)
shapiro.test(post)

#one-sample t-test: 单样本t检验
t.test(pre, mu=5) #mu假定该数据平均值为5

#paired-sample t-test: 配对样本t检验
t.test(pre,post,alternative = "two.sided",paired = TRUE)

#independent-sample t-test: 独立样本t检验
#方差一致性检验：检验方差是否一致
var.test(pre,post,alternative = "two.sided")
```

预习任务

院（系）：医疗器械学院

日期：2023.10.08

t 检验&方差检验（预习）

实验任务：

- 1、预习单样本t检验
- 2、预习配对样本t检验
- 3、预习两独立样本t检验
- 4、预习方差齐性检验

实验工具：

电脑、Internet

预习内容：**t 检验（t test）****【定义】**

• t 检验，又称学生 t 检验（Student t-test）可以说是统计推断中非常常见的一种检验方法，用于统计量服从正态分布，但方差未知的情况。

【常见 4 个用途】**1. 单样本均值检验（One-sample t-test）**

• 用于检验 总体方差未知、正态数据或近似正态的 单样本的均值 是否与 已知的总体均值相等。

目的：

- 检验单样本的均值是否和已知总体的均值相等。

要求：

- 1) 总体方差未知，否则就可以利用 Z 检验（也叫 U 检验，即正态检验）
- 2) 正态数据或近似正态

2. 两独立样本均值检验（Independent two-sample t-test）

• 用于检验 两对独立的 正态数据或近似正态的 样本的均值 是否相等，这里可根据总体方差是否相等分类讨论

目的：

- 检验两独立样本的均值是否相等。

要求：

- 两样本独立，服从正态分布或近似正态。

3. 配对样本均值检验（Dependent t-test for paired samples）

- 用于检验 一对配对样本的均值的差 是否等于某一个值

要求：

- 总体方差相等，正态数据或近似正态

4. 回归系数的显著性检验 (t-test for regression coefficient significance)

- 用于检验 回归模型的解释变量对被解释变量是否有显著影响

目的:

• 检验回归模型的回归系数是否等于给定的值，一般取为 0，此时检验的意义是检验该回归系数对应的解释变量对被解释变量是否有显著影响（因为若接受取值为 0 的假设，则该解释变量的项对被解释变量没有作用了）。

方差齐性检验:

【定义】

• 方差齐性检验 (Equality of variance test) 是指检验两个样本的方差是否相等的统计检验方法。

常用方法:

- 把样本的方差分别求出来，然后比较它们是否相等。

意义:

• 做两独立样本 T 检验，如果两个样本的方差不相等，可能会导致 T 检验结果出现偏差，也就是显著性水平判断错误，此时进行方差齐性检验。

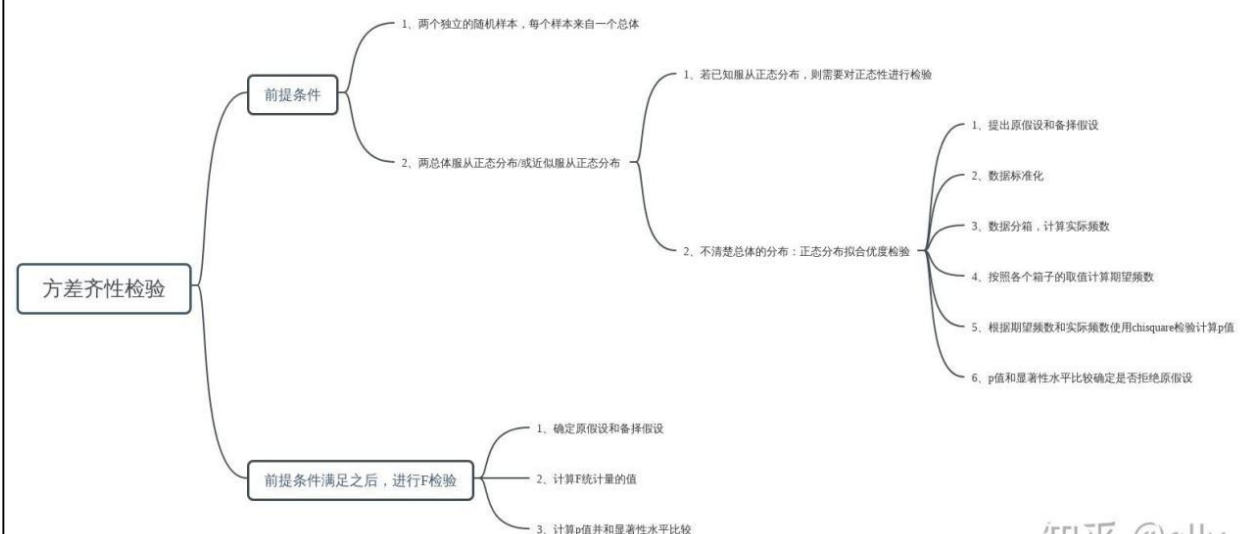
实训总结:

t 检验小结:

不管单样本 t 检验、配对样本 t 检验还是两独立样本 t 检验，都是用于检验两个总体间计量资料的比较方法。单样本 t 检验要求符合正态分布，两独立样本 t 检验要求独立、正态和方差齐，配对 t 检验要求差值符合正态分布。

上述三条对正态分布的要求不是非常严格，近似正态分布依然可以分析，也可以采用非参数检验的方法进行分析。

方差齐性检验:



实验任务二

院（系）：医疗器械学院

日期：2023.10.20

方差分析（验证性实验）

实验任务：

- 1、掌握完全随机设计的方差分析
- 2、掌握随机区组设计的方差分析
- 3、掌握多样本均数的两两比较分析
- 4、掌握方差齐性检验

实验工具：

电脑、Internet、R、Python

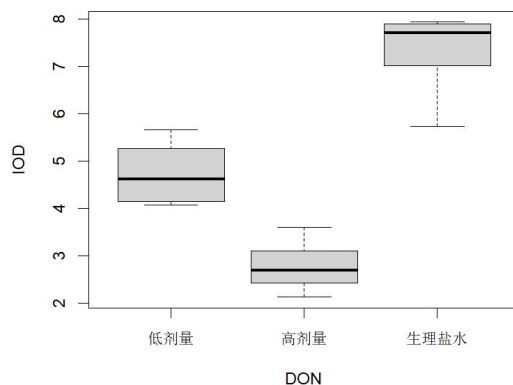
实验过程：

1. 8-1

运行代码：

```
1. setwd("E:\\AAAAAAAAAAAAAAAAAAAA\\sophomore\\subject\\医疗大数据统计\\Data\\8")
2. #设置读取文件路径
3. data8.1<-read.csv("8-1.csv",fileEncoding = "GBK")
4. #plot 函数主要用于绘制散点图和折线图
5. plot(IOD~DON,data = data8.1,type="p")
6. #boxplot 函数是绘制箱型图的，箱型图是一类用来展示数据分布范围的图形
7. #以得到一组数据的上限值、下限值、上下四分位值、以及中位数和异常值再绘制箱型图
8. boxplot(IOD~DON,data = data8.1)
9. #aov()函数（单因素方差分析）的语法为 aov(formula,data=dataframe)
10. aov.data8.1 <- aov(IOD~DON,data = data8.1)
11. # summary(): 获取描述性统计量，可以提供最小值、最大值、四分位数和数值型变量的均值
12. summary(aov.data8.1)
```

结果截图：



```
      Df Sum Sq Mean Sq F value    Pr(>F)
DON      2   84.81    42.41    103.1 1.4e-11 ***
Residuals 21    8.65     0.41
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

2. 8-2

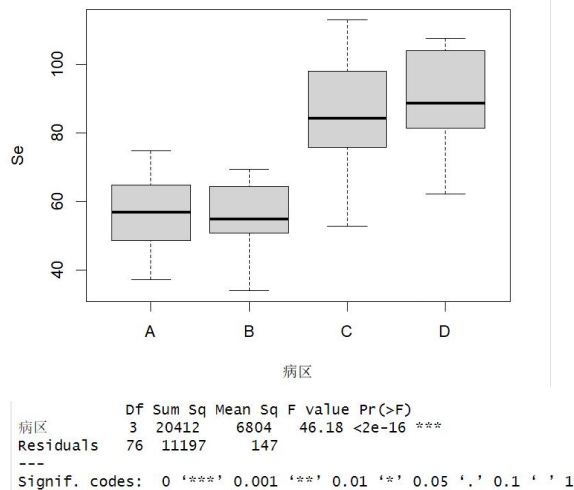
运行代码:

```

1. #8-2
2. data8.2<-read.csv("8-2.csv",fileEncoding = "GBK")
3. plot(Se~ 病区,data = data8.2)
4. boxplot(Se~ 病区,data = data8.2)
5. aov.data8.2<-aov(Se~ 病区,data = data8.2)
6. summary(aov.data8.2)

```

结果截图:



3. 8-4

运行代码及结果:

```

> #8-4
> data8.4<-read.csv("8-4.csv")
> quzu<-gl(12,1,36)
> chuli<-gl(3,12)
> data8.4<-data.frame(X=data8.4$X,quzu,chuli)
> aov.data8.4<-aov(X~quzu+chuli,data = data8.4)
> summary(aov.data8.4)

```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
quzu	11	733.8	66.7	2.735	0.0214 *
chuli	2	722.7	361.4	14.818	8.4e-05 ***
Residuals	22	536.5	24.4		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

实训总结:

Plot 函数:

- plot 函数主要用于绘制散点图和折线图

Boxplot 函数:

• boxplot 函数是绘制箱型图的,箱型图是一类用来展示数据分布范围的图形,以得到一组数据的上限值、下限值、上下四分位值、以及中位数和异常值再绘制箱型图

Aov() 函数:

- `aov()` 函数（单因素方差分析）
- 语法为 `aov(formula, data=dataframe)`

Summary 函数：

• `summary()`：获取描述性统计量，可以提供最小值、最大值、四分位数和数值型变量的均值

`gl()`：

- `gl()` 在 R 语言中的函数用于通过指定其级别的模式来生成因子
- `gl()` (级别数, 重复次数, 结果长度, 标签, 对级别进行排序的布尔值)

【方差定义】

方差分析 (Analysis of Variance, 简称 ANOVA) 是一种统计方法, 用于比较三个或三个以上组 (或处理) 之间的均值是否存在显著差异。

【常见类型】

1. **单因素方差分析 (One-Way ANOVA)**：用于比较一个因素对于一个连续变量的影响, 例如比较不同药物剂量对于治疗效果的影响。
2. **双因素方差分析 (Two-Way ANOVA)**：用于比较两个因素对于一个连续变量的影响, 通常包括两个独立变量, 例如考察不同肥料类型和不同浇水频率对于植物生长的影响。
3. **多因素方差分析 (Multifactor ANOVA)**：用于比较多个因素对于一个连续变量的影响, 可以包括多个独立变量, 以考察多个因素的联合影响。

【方差分解】

方差分析的主要思想是通过将总体方差分解为组内方差和组间方差来检测组均值之间的显著性差异。

实验任务三

院（系）：医疗器械学院

日期：2023.12.26

正态分布（验证性实验）

实验任务：

1、掌握正态分布

实验工具：

电脑、Internet、R、Python

实验过程：

1. 9-11

运行代码及结果截图：

```
> setwd("E:\\AAAAAAAAAAAAAAAAAAAA\\sophomore\\subject\\医疗大数据统计\\Data\\9")
> #9-11
> data9.11<-read.csv("9-11.csv",sep="," ,header=F)
> mean<-mean(data9.11$V1) #平均值
> sd<-sd(data9.11$V1) #标准偏差
> A<-table(cut(data9.11$V1,br=c(27,39,51,63,75,89)))
> ##table() 可以用来统计每个因子频数
> #cut() 函数通过将一个连续变量分割后形成因子变量
> Pr<-pnorm(c(28,40,52,64,76,89),mean,sd)
> #pnorm () 该函数给出正态分布随机数的概率小于给定数的值
> p<-c(Pr[2]-Pr[1],Pr[3]-Pr[2],Pr[4]-Pr[3],Pr[5]-Pr[4],Pr[6]-Pr[5])
> t<-length(data9.11$V1)*p
> Xsquad<-sum((A-t)^2/t)
> Xsquad
[1] 2.510309
```

实训总结：

Cut 函数：

- cut() :将对应的数据划分到对应区间
- cut(x, y, labes, right, include.lowest)
- x:待划分的数据
- y:判断依据（区间）
- right:逻辑值,默认为区间左开右闭；若为 FALSE,则左闭右开。（知识补充：开区间是不包括端点(a,b)，闭区间是包括端点[a,b]。）
- include.lowest:逻辑值,include.lowest=TRUE,第一个区间包含左端点,最后一个区间包含右端点。

Table 函数：

- table() 可以用来统计每个因子频数
- Table()函数，可用于计算多维频数表。频数表是一种用于展示数据中各个类别的频次统计的表格。Table()函数可用来了解数据的分布情况，发现模式和关联性。

Pnorm():

- pnorm() 该函数给出正态分布随机数的概率小于给定数的值。 它也被称为“累积分布函数”。

- 翻译一下就是：返回值是正态分布的分布函数值，比如 pnorm(z) 等价于 $P[X \leq z]$

- pnorm(x, mean, sd)

x 是数字的向量。

p 是概率的向量。

n 是观察的数量（样本大小）。

mean 是样本数据的平均值。 它的默认值为零。

sd 是标准偏差。 它的默认值为 1。

实验任务四

院（系）：医疗器械学院

日期：2023. 12. 26

秩和检验（一）（验证性实验）

实验任务：

- 1、掌握配对资料符号秩和检验
- 2、掌握两独立样本比较的秩和检验

实验工具：

电脑、Internet、R、Python

实验过程：**设置读取文件路径**

```
setwd("E:\\AAAAAAAAAAAAAAAAAAAA\\sophomore\\subject\\医疗大数据统计\\Data\\10")
```

1. 10-1

运行代码及结果截图：

```
> #10-1
> A<-read.csv("10-1.csv",fileEncoding = "GBK")
> median(ASDON) # median() 函数用来计算样本的中位数
[1] 142.8

> #执行wilcoxon秩和检验验证数据分布是否一致
> #函数及格式: wilcox.test(y~x,data) y是连续变量, x是一个二分变量
> ##exact表示是否算出精确的p值, correct表示大样本时是否做连续型修正
> zhihejianyan1<-wilcox.test(ASDON~A$中位数,exact=FALSE,correct=FALSE)
> zhihejianyan1
```

Wilcoxon signed rank test

```
data: ASDON ~ A$中位数
V = 184, p-value = 0.003132
alternative hypothesis: true location is not equal to 0
```

2. 10-2

运行代码及结果截图：

```
> #10-2
> B<-read.csv("10-2.csv",fileEncoding = "GBK")
> zhihejianyan2<-wilcox.test(B$ 放射免疫法,B$ 酶联免疫法,exact=FALSE,paired=TRUE,correct=FALSE)
> zhihejianyan2

      Wilcoxon signed rank test

data:  B$放射免疫法 and B$酶联免疫法
V = 21.5, p-value = 0.9054
alternative hypothesis: true location shift is not equal to 0
```

3. 10-3

运行代码及结果截图：

```
> #10-3
> C<-read.csv("10-3.csv",fileEncoding = "GBK")
> zhihejianyan3<-wilcox.test(Ca含量 ~ 性别,data=C,exact=FALSE,correct=FALSE,paired=FALSE)
> zhihejianyan3

      Wilcoxon rank sum test

data:  Ca含量 by 性别
W = 78, p-value = 0.03429
alternative hypothesis: true location shift is not equal to 0
```

实训总结：

Median() 函数：

- median() 函数用来计算样本的中位数
- 中位数 (Median) 又称中值，统计学中的专有名词，是按顺序排列的一组数据中居于中间位置的数，代表一个样本、种群或概率分布中的一个数值，其可将数值集合划分为相等的上下两部分

- **语法格式：** median(x, na.rm = FALSE)
- x 输入向量
- na.rm 布尔值，默认为 FALSE，设置是否删除输入的向量中的缺失值 NA，设置 TRUE 删除 NA
- median 函数的输入向量中，如果元素没有值，则默认为 NA，可以通过第三个参数来设置是否删除默认的 NA 值，如果没有删除 NA 返回结果为 NA

Wilcoxon 检验函数：

- 执行 wilcoxon 秩和检验验证数据分布是否一致
- **函数格式：** wilcox.test(y~x,data)
- y 是连续变量，x 是一个二分变量
- exact 表示是否算出精确的 p 值

- `correct` 表示大样本时是否做连续型修正

#函数1:

```
wilcox.test(x, y = NULL, #x一组数据, y一组数据
            alternative = c("two.sided", "less", "greater"), #备择假设方向
            mu = 0, #单样本检验时的
            exact = NULL, #是否精确
            correct = TRUE, #是否校正
            paired = FALSE, #是否为配对样本
            var.equal = FALSE, #方差是否齐
            conf.level = 0.95, #置信区间
            tol.root = 1e-4,
            digits.rank = Inf, ..)
```

#函数2:

```
wilcox.test(formula, #格式x~y, x为数值变量, y为分组
            data, #数据集名称
            subset, na.action, ...)
```

实验任务五

院（系）：医疗器械学院

日期：2023.12.26

秩和检验（二）（验证性实验）

实验任务：

- 1、掌握配对资料符号秩和检验
- 2、掌握两独立样本比较的秩和检验
- 3、掌握多独立样本比较的秩和检验

实验工具：

电脑、Internet、R、Python

实验过程：

1. 10-4

运行代码及结果截图：

```
> #10-4
> D<-read.csv("10-4.csv",fileEncoding = "GBK")
> zhihejianyan4<-wilcox.test(分值 ~ 地区,data=D,exact=FALSE,correct=FALSE,paired=FALSE)
> zhihejianyan4

      Wilcoxon rank sum test

data:  分值 by 地区
W = 3255, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
```

2. 10-5

运行代码及结果截图：

```
> #10-5
> E<-read.csv("10-5.csv",fileEncoding = "GBK")
> zhihejianyan5<-wilcox.test(临床分度 ~ 地区,data=E,exact=FALSE,correct=FALSE,paired=FALSE)
> zhihejianyan5

      Wilcoxon rank sum test

data:  临床分度 by 地区
W = 5355.5, p-value = 0.8835
alternative hypothesis: true location shift is not equal to 0
```

3. 10-6

运行代码及结果截图：

```
> #10-6
> F<-read.csv("10-6.csv",fileEncoding = "GBK")
> # kruskal.test检验：用于比较多个独立样本的中位数是否相等
> #适用于数据不满足正态分布或方差不齐的情况
> zhihejianyan6<-kruskal.test(TNF ~ 组别,data=F)
> zhihejianyan6

Kruskal-wallis rank sum test

data: TNF by 组别
Kruskal-wallis chi-squared = 11.18, df = 2, p-value = 0.003735
```

实训总结：

kruskal.test 检验：

• 用来解决多独立样本难以满足方差分析条件（独立性、正态性、方差齐性）时统计推断问题

• Kruskal-Wallis 检验是一种用于比较多个独立样本的中位数差异的方法。它适用于非正态分布的数据，或者在正态分布假设不满足的情况下。

• Kruskal-Wallis 检验的原假设是各组中位数相等，备择假设是至少有一组中位数不同。

• 该检验基于秩和的比较，而不是原始数据的数值。因此，它被称为非参数检验。

• **函数格式：**kruskal.test(y ~ A, data)

• y 为连续变量，A 为两个或更多水平的分组变量

```
kruskal.test(formula, data, subset, na.action, ...)
```

formula这里格式为response ~ group, response 观测值, group分组变量。

实验任务六

院（系）：医疗器械学院

日期：2023.12.26

相关系数检验（验证性实验）

实验任务：

- 1、掌握两个变量间的相关系数
- 2、掌握相关系数检验

实验工具：

电脑、Internet、R、Python

实验过程：

1. 11-1

运行代码及结果截图：

```
> setwd("E:\\AAAAAAAAAAAAAAAAAAAA\\sophomore\\subject\\医疗大数据统计\\D
ata\\11")
> data11.1<-read.csv("11-1.csv")
> #cor()用于计算两个向量之间的相关系数
> xgxs<-cor(data11.1$OAP,data11.1$DON)
> xgxs
[1] 0.7863221
```

```
> # 添加显著性标记:
> # 使用cor.mtest做显著性检验;
> cor.test(data11.1$OAP,data11.1$DON)
```

Pearson's product-moment correlation

```
data: data11.1$OAP and data11.1$DON
t = 7.6365, df = 36, p-value = 4.89e-09
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.6233270 0.8838328
sample estimates:
      cor
0.7863221
```

实训总结：**Cor 函数：**

- cor()用于计算两个向量之间的相关系数
- 相关系数是用于衡量两个变量之间线性关系强度的指标，取值范围为-1 到 1 之间
- 相关系数为-1，两个变量之间完全负相关
- 相关系数为 1，两个变量之间完全正相关
- 相关系数为 0，两个变量之间不存在线性关系
- **语法格式：** cor(x, y = NULL, use = "everything", method =


```
c(“pearson”, “kendall”, “spearman”))
```

- x: 数字向量、矩阵或数据帧。
- y: NULL (默认值) 或向量、矩阵或与 x 兼容的数据帧。默认值相当于 $y = x$ (但更有效)。
- use: 一个可选字符串, 提供在缺失值存在时计算协方差的方法, 其必须是字符
- method: 指示要计算的相关系数(或协方差)的字符串

Cor.test 检验:

- 相关性检验函数
- **函数格式:** `cor.test(x, y, alternative, method)`
- x, y 表示向量;
- alternative 指定单侧检验还是双侧检验
- two.sided 同时检验正负相关, greater 检验正相关, less 检验负相关;
- method 指定使用哪种相关性衡量指标(pearson、kendall 还是 spearman)

实验任务七

院（系）：医疗器械学院

日期：2023. 12. 23

生存分析（一）（验证性实验）

实验任务：

- 1、掌握生存分析常用库
- 2、掌握生存分析函数

实验工具：

电脑、Internet、R、Python

实验过程：**设置读取文件路径**

```
setwd("E:\\AAAAAAAAAAAAAAAAAAAA\\sophomore\\subject\\医疗大数据统计\\Data\\12")
```

1. 12-1

运行代码及结果截图：

```
> #12-1
> library(survival) #载入survival包
> # survival库是一个用于生存分析的常用库，包含Surv函数。
> data12.1<-read.csv("12-1.csv")
> # surv函数：用于构建生存分析模型中的生存时间和事件发生的数据格式
> Surv(data12.1$scsj,data12.1$zt)
[1] 2 5 8 9 9+ 10 13 13 15+ 18 20 23+

> fit<-survfit(Surv(data12.1$scsj,data12.1$zt)~1,data=data12.1)
> #Survfit函数：用于计算生存曲线和生存概率
> #通过增加一个因子变量，可以根据该变量的不同水平来比较不同组别之间的生存情况
> summary(fit) #summary():方便查看模型的汇总给统计信息以及解读特征
Call: survfit(formula = Surv(data12.1$scsj, data12.1$zt) ~ 1, data = data12.1)
```

time	n.risk	n.event	survival	std.err	lower	95% CI upper	95% CI
2	12	1	0.917	0.0798	0.7729	1.000	
5	11	1	0.833	0.1076	0.6470	1.000	
8	10	1	0.750	0.1250	0.5410	1.000	
9	9	1	0.667	0.1361	0.4468	0.995	
10	7	1	0.571	0.1462	0.3461	0.944	
13	6	2	0.381	0.1470	0.1789	0.811	
18	3	1	0.254	0.1426	0.0845	0.764	
20	2	1	0.127	0.1147	0.0216	0.745	

2. 12-2

运行代码:

```

1. #12-2
2. data12.2<-read.csv("12-2.csv")
3. colnames(data12.2)[1]<-c("xh") #colnames: 指定矩阵的列名称
4. data12.2$qcjzrs<-data12.2$qcrs-data12.2$ssls/2
5. data12.2$q<-data12.2$swls/data12.2$qcjzrs
6. data12.2$p<-1-data12.2$q
7. data12.2$s[1]<-data12.2$p[1]
8.
9. for(i in 2:22){
10.   data12.2$s[i]<- data12.2$s[i-1]*data12.2$p[i]
11. }
12.
13. x<-data12.2$q/(data12.2$p*data12.2$qcrs)
14. y<-c(x[1],rep(NA,21))
15. for(i in 2:22){
16.   y[i]<-sum(x[1:i])
17. } #循环求和
18.
19. for(i in 1:22){
20.   data12.2$se[i]<-data12.2$s[i]*sqrt(y[i])
21. } #循环求方差
22.
23. data12.2

```

结果截图:

	xh	qzns	swls	ssls	qcrs	qcjzrs	q	p	s
1	1	0~	51	0	1166	1166.0	0.04373928	0.9562607	0.95626072
2	2	2~	45	0	1115	1115.0	0.04035874	0.9596413	0.91766724
3	3	4~	66	1	1070	1069.5	0.06171108	0.9382889	0.86103700
4	4	6~	56	1	1003	1002.5	0.05586035	0.9441397	0.81293917
5	5	8~	65	20	946	936.0	0.06944444	0.9305556	0.75648507
6	6	10~	61	26	861	848.0	0.07193396	0.9280660	0.70206810
7	7	12~	75	27	774	760.5	0.09861933	0.9013807	0.63283061
8	8	14~	67	0	672	672.0	0.09970238	0.9002976	0.56973589
9	9	16~	59	2	605	604.0	0.09768212	0.9023179	0.51408288
10	10	18~	43	19	544	534.5	0.08044902	0.9195510	0.47272542
11	11	20~	61	19	482	472.5	0.12910053	0.8708995	0.41169632
12	12	22~	56	43	402	380.5	0.14717477	0.8528252	0.35110501
13	13	24~	34	30	303	288.0	0.11805556	0.8819444	0.30965511
14	14	26~	25	22	239	228.0	0.10964912	0.8903509	0.27570170
15	15	28~	30	17	192	183.5	0.16348774	0.8365123	0.23062785
16	16	30~	24	17	145	136.5	0.17582418	0.8241758	0.19007790
17	17	32~	11	15	104	96.5	0.11398964	0.8860104	0.16841099
18	18	34~	13	19	78	68.5	0.18978102	0.8102190	0.13644978
19	19	36~	6	24	46	34.0	0.17647059	0.8235294	0.11237041
20	20	38~	5	5	16	13.5	0.37037037	0.6296296	0.07075174
21	21	40~	1	1	6	5.5	0.18181818	0.8181818	0.05788779
22	22	42~	1	3	4	2.5	0.40000000	0.6000000	0.03473267

	se
1	0.005989281
2	0.008049700
3	0.010130041
4	0.011421781
5	0.012574246
6	0.013437067
7	0.014258602
8	0.014774344
9	0.015000312
10	0.015039977
11	0.014956376
12	0.014683823
13	0.014493826
14	0.014342102
15	0.014074032
16	0.013700513
17	0.013506902
18	0.013254161
19	0.013340290
20	0.015955805
21	0.017162093
22	0.017524075

实训总结：**Survival 包：**

- “survial”包可以针对单样本或者多样本进行生存分析，可以使用的模型有参数加速失效模型和 Cox 比例风险模型
- 生存分析的数据一般包括三个部分：起始时间、结束时间和结束时患者的状态
- 通常 1 代表目标事件发生，0 代表删失数据，即目标事件未发生或者失访等
- 这里的目标事件一般是指疾病或者死亡
- 另外，数据也可以是到结束时所经历的时间段和结束时患者的状态
- 通常，使用 Surv() 函数来将数据进行格式转化，便于进行后续分析。

Surv 函数：

- 用于构建生存分析模型中的生存时间和事件发生的数据格式

以下是使用R语言中的 Surv 函数的整个流程：

步骤	描述
步骤一	导入所需的库
步骤二	准备数据
步骤三	使用 Surv 函数创建生存时间和事件发生的数据格式
步骤四	使用生存数据进行生存分析

Survfit 函数：

- 用于计算生存曲线和生存概率
- 通过增加一个因子变量，可以根据该变量的不同水平来比较不同组别之间的生存情况
- 函数格式：`survfit(surv(time, status) ~ 1, data=your_data)`
- time 是生存时间变量，status 是时间状态变量，your_data 是数据集名称

Summary 函数：

- 获取描述性统计量，可以提供最小值、最大值、四分位数和数值型变量的均值，以及因子向量和逻辑型向量的频数统计等
- 方便查看模型的汇总统计信息以及解读特征

Colnames 函数：

- 用来指定矩阵的列名称
- 其允许为每一列设置一个有意义的名称，以便更好地理解和处理数据

实验任务八

院（系）：医疗器械学院

日期：2023.12.26

生存分析（二）（验证性实验）

实验任务：

- 1、掌握生存分析库
- 2、掌握条件语句函数
- 3、掌握生存曲线的时序检验

实验工具：

电脑、Internet、R、Python

实验过程：

1. 12-3

运行代码及结果截图：

```
> #12-3
> library(survival)
> data12.3<-read.csv("12-3.csv")
> #ifelse函数：条件语句函数
> data12.3$zb<-ifelse(data12.3$zb==1,"辅助化疗","单纯手术")
> survdiff(Surv(scsj,zt)~zb,data=data12.3) # survdiff函数：进行生存曲线的时序检验
Call:
survdiff(formula = Surv(scsj, zt) ~ zb, data = data12.3)

              N Observed Expected (O-E)^2/E (O-E)^2/V
zb=单纯手术 10          9      5.11      2.95      4.99
zb=辅助化疗 12          9     12.89      1.17      4.99

Chisq= 5  on 1 degrees of freedom, p= 0.03
```

```
> alpha<-0.05
> qchisq(1-alpha/2,df=1,lower.tail = TRUE)
[1] 5.023886
```

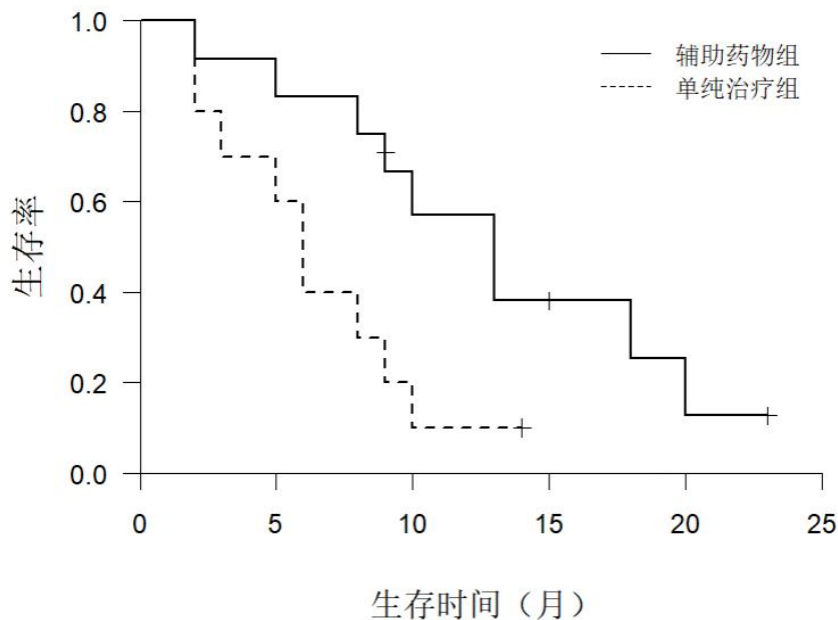
2. 12-5

运行代码：

```
1. #12-5
2. library(survival)
3.
4. #####
5. data12.5<-read.table("12-5.csv",sep=",")
6. hanshu<-survfit(Surv(data12.5$V1,data12.5$V2)~data12.5$V3)
7.
8. plot(hanshu, xlab=" 生存时间（月）",ylab="",lty=1:2,
9.       mark.time=TRUE,mark=3,conf.int=F,lwd=1.5,cex.lab=1.2,
10.      xlim=c(0,25),ylim=c(0,1),axes=F)
11.
12. legend(16,1,c("辅助药物组","单纯治疗组"),bty="n",lty = 1:2) #标签
13.
14. mtext("生存率",side=2,2,font=2,cex=1.2) #纵坐标标签
```

```
15. #####
16.
17. axis(2,pos=c(0),at=seq(from=0,to=1,by=0.2),las=1) #纵坐标
18. axis(1,pos=c(0),at=seq(from=0,to=25,by=5)) #横坐标
```

结果截图：



实训总结：

Ifelse 函数：

- 条件语句函数
- **函数格式：** ifelse(condition, true_value, false_value)
- 第一个参数 condition 是一个逻辑表达式，它可以是任何可以返回 TRUE 或 FALSE 的表达式
- 第二个和第三个参数分别是在 condition 为 TRUE 和 FALSE 时返回的值
- 应用：
 - ① ifelse 函数可以用于数据清洗中。例如，可以使用 ifelse 函数将数据中的缺失值替换为一个默认值。
 - ② ifelse 函数可以用于根据已有字段创建新字段，比如根据某个条件创建一个二元变量
 - ③ ifelse 函数可以用于对数据进行变换，例如根据数值大小对数据进行分组划分
- 注意事项：
 - I 当 condition、true_value 和 false_value 的长度不同时，ifelse 函数会自动重复短的值
 - II ifelse 函数在处理大型数据时可能会变得非常慢，因为它在执行时需要遍历整个数据集

Survdiff() 函数:

- 用于进行生存曲线的时序检验
- **函数格式:** `survdiff(surv_object ~ group, data = data)`

```
1 # 创建生存对象
2 surv_object <- Surv(time, event)
3
4 # 使用survdiff函数执行时序检验
5 result <- survdiff(surv_object ~ group, data = data)
```

- time 代表生存时间的向量
- event 代表时间发生情况的向量
- group 代表组别的向量
- data 代表包含这些变量的数据框

实验评阅

实验任务：拓展感兴趣知识（思考性实验）

实验任务一：t 检验（验证性实验）

预习任务：t 检验&方差检验（预习）

实验任务二：方差分析（验证性实验）

实验任务三：正态分布（验证性实验）

实验任务四：秩和检验（一）（验证性实验）

实验任务五：秩和检验（二）（验证性实验）

实验任务六：相关系数检验（验证性实验）

实验任务七：生存分析（一）（验证性实验）

实验任务八：生存分析（二）（验证性实验）